Michael Ji

647-938-0623 | michael.jw.ji@gmail.com | linkedin.com/in/michael-jw-ji | github.com/michael-jw-ji | michael-ji.vercel.app

Education

University of Waterloo — Waterloo, ON
Bachelor of Computing and Financial Management (Honors, Co-op) — *Expected April 2028*

Experience

Software Engineer Intern — September 2026 – Present
Communitech (Data & AI Automation @ Langdon Hall) — *Waterloo, ON*

- Building **Python/SQL** ingestion and aggregation pipelines on **Microsoft Fabric**, transforming reservation, spa, restaurant, and POS data from **4 source systems** into a unified guest and revenue model for cross-department analytics.
- Built a **Claude-powered** report generator on top of the unified guest model, turning data from 4 systems into daily and per-guest department summaries via a curated system prompt, cutting lookup time from **30 to 2 minutes**.
- Hardening the platform for guest PII with row-level security, least-privilege **Entra ID** service principals per pipeline, secrets in **Azure Key Vault**, and Canadian-region data residency for **PIPEDA** compliance.

Software Engineer Intern — May 2026 – August 2026
Enns Real Estate (Full-Stack Deployment) — *Waterloo, ON*

- Designed and built the brokerage's listings platform end to end on **Next.js/TypeScript** and **Supabase**, from schema design to deployment, contributing to **200% year-over-year revenue growth** after launch with 200+ monthly visitors.
- Reduced listing page load time by **40%** by caching listing queries with **Next.js** incremental static regeneration and indexing Postgres filter columns for price, bedrooms, and location, revalidating pages on every listing update.
- Automated staging and production releases with **GitHub Actions** and **Vercel CI/CD**, gating every PR on typecheck, lint, and build checks, cutting release time to **under 5 minutes** with **0 failed production deploys.**

Software Engineer Intern — January 2026 – April 2026
Rogers Communications (AI Governance Platform) — *Toronto, ON*

- Built **Python** APIs and document ingestion pipelines for an internal policy assistant serving **5K+ users**, indexing 500+ policy documents into **Azure AI Search**, handling **35K+ weekly queries** at 3s p95 latency with **Azure OpenAI**.
- Designed request-validation middleware in front of the RAG APIs with LLM-based query normalization, prompt-injection detection, oversized-input rejection, and file-type allowlisting, blocking **95%** of 250+ adversarial test cases.
- Shipped an async pipeline alerting owners on suspicious prompts via Azure OpenAI function calling and Power Automate, decoupled from the response path to keep user **p95 latency at 3s**, cutting review time from **2 hours to 5 minutes**.

Machine Learning Research Intern — April 2025 – March 2026
Algoverse (Model Interpretability) — *Palo Alto, CA*

- Trained per-layer probes reaching **0.98** macro-F1 with five-seed confidence intervals and random-label controls, showing near-identical probe scores can reflect lexical shortcuts; **2 papers accepted at NeurIPS 2026 workshops.**
- Built a **Python** dataset pipeline labeling 5 variable roles across 7 programming languages using AST and regex extractors, with problem-hash 70/10/20 splits to prevent train/test leakage; published as a public **Hugging Face** dataset.
- Designed a model-agnostic experiment CLI extracting per-layer hidden states from any **Hugging Face** LLM, running perturbation and cross-language probe sweeps across 3 models and 5 variable roles with resumable, checkpointed runs.

Projects

Serverless Filing Ingestion and Query Platform | *Python, AWS Lambda, DynamoDB, EventBridge, SQS, Idempotency*

- Built an event-driven ingestion service for Canadian insider filings on **AWS Lambda** and **EventBridge** with scheduled polling, batched parsing, idempotent writes, and **SQS** dead-letter retries, processing **10K+ filings** with **0 duplicates**.
- Designed a **DynamoDB** schema around access patterns with issuer/date composite keys and a secondary index for insider lookups, serving per-company and per-insider history in single queries with p95 reads under **300ms**.

Badminton Match Analytics Platform | *Python, Flask, PostgreSQL, Docker, REST APIs, Schema Design*

- Built a match analytics platform for Waterloo Warriors players with **Flask** REST APIs and a normalized **PostgreSQL** schema for players, matches, sets, and season aggregates, serving **100+ active users** across **1,000+ logged matches**.
- Designed **REST** endpoints for match histories, player profiles, and season stats with request validation and pagination, backed by an ingestion service deduplicating Athletics results, deployed with **Docker** on Heroku.
- Built **OpenAI**-powered opponent scouting that summarizes teammate match notes into per-opponent weakness reports, with per-user rate limiting and cached summaries invalidated on new notes, cutting API cost by **25%**.

Skills

Languages: Python, C/C++, TypeScript/JavaScript, SQL, Java, Go, Rust
Databases & Frameworks: PostgreSQL, DynamoDB, Supabase, Microsoft Fabric; FastAPI, Flask, Next.js, React.js, Node.js
Technologies: AWS, Azure, Docker, GitHub Actions, Vercel, Git, REST APIs, Linux, Kubernetes, Kafka